Audspect BAS — Developer Guide

Classification: Internal — Audspect Engineering / Confidential
Platform Version: v1.7.5

Repository Layout

```
Audspect_Cloud/
├── orchestrator/                # Go orchestrator (primar
│   ├── cmd/server/              # Main entrypoint + wwwro
│   │   └── wwwroot/             # Windows Junction → orch
│   ├── wwwroot/                 # Actual static files: in
│   ├── internal/                 # ~50 packages, organize
│   │   ├── api/                  # HTTP handler functions
│   │   ├── auth/                 # Password hashing, JWT,
│   │   ├── db/                   # Schema (EnsureSchema)
│   │   ├── integrity/             # Scenario signing, bin
│   │   ├── models/               # DB row structs, WS mes
│   │   ├── connector/             # MISP/OpenCTI/OTX thre
│   │   ├── dashboard/              # Executive dashboard
│   │   ├── threatpriority/         # Per-actor threat pri
│   │   ├── jobs/                   # Fleet job engine (sc
│   │   ├── campaign/               # Campaign fan-out (ag
│   │   └── reporting/              # HTML/PDF/CSV/JSON re
│   └── config/                   # Config loading (env va
├── agent/                       # Go agent binary (Windows
├── scenarios/                   # Signed scenario YAML fi
├── docs/                        # Documentation
│   ├── guides/                  # Customer sub-guides
```

```
│   └── internal/                # This set of internal do
└── packaging/
    ├── windows-build.ps1        # Build pipeline (Windows
    └── compose/install.sh       # Customer-facing install
```

`internal/` has grown to roughly 50 packages as the platform expanded — the table above is a representative subset, not exhaustive. When looking for where something lives, `grep -rl` for the feature name across `internal/` is usually faster than guessing from this list.

Important: `orchestrator/cmd/server/wwwroot` is a Windows Junction pointing to `orchestrator/wwwroot`. Only edit files in `orchestrator/wwwroot/`. The two source files are:

- `orchestrator/wwwroot/index.html` — primary dashboard
- `orchestrator/wwwroot/og/index.html` — legacy dashboard (kept for backward compat)

The build script is `packaging/windows-build.ps1`, not `windows-build.ps1` at the repo root — that path doesn't exist. Easy to get wrong from memory/habit; always check the actual path.

Development Environment

Prerequisites

- Go 1.22+
- PostgreSQL 16 (via Docker or local install)
- Docker (for running dependencies)
- `garble` (for production builds only; dev builds use `go build`)

Start development database

```
docker run -d \
```

```
  --name bas-postgres-dev \
  -e POSTGRES_USER=bas \
  -e POSTGRES_PASSWORD=basdev \
  -e POSTGRES_DB=bas \
  -p 5432:5432 \
  postgres:16
```

Configure environment

```
export DATABASE_URL="postgres://bas:basdev@localhost:5432
export JWT_SECRET="dev-jwt-secret-minimum-32-characters"
export AGENT_SECRET="dev-agent-secret"
export HTTP_PORT=9000
export BAS_ADMIN_EMAIL="admin@dev.local"
export BAS_ADMIN_PASSWORD="DevPassword1!"
export SCENARIOS_DIR="scenarios"
```

Run the orchestrator (dev mode, no garble)

```
cd orchestrator
go run ./cmd/server/...
```

Dashboard available at `http://localhost:9000`.

Key Architectural Decisions

Agent is a dumb executor

The agent receives fully-formed commands (ready to execute in PowerShell/Bash). All intelligence — ART YAML parsing, technique resolution, Caldera ability fetching, output interpretation — happens

server-side. The agent has zero knowledge of ART, Caldera, or the ATT&CK framework.

Why: Simplifies agent updates (logic stays on server), reduces agent binary complexity, makes the agent harder to reverse-engineer for adversaries inspecting it on endpoints.

Garble breaks reflection — use JSON-tag maps

The orchestrator binary is built with `garble -literals -tiny`. Garble obfuscates struct field names, breaking Go's `html/template` when templates reference struct fields by name (e.g., `{{.FieldName}}`).

Pattern to follow: Never pass structs to `html/template.Execute()`. Always convert to `map[string]interface{}` using JSON marshaling and the JSON tag names:

```
// Wrong — field names obfuscated by garble at runtime
tmpl.Execute(w, reportData)

// Correct — JSON tags are strings (not symbol names), su
jsonBytes, _ := json.Marshal(reportData)
var m map[string]interface{}
json.Unmarshal(jsonBytes, &m)
tmpl.Execute(w, m)
```

Result delivery is at-least-once + idempotent

Agents retry result submission on failure. The server REPLACES results for a run step (never appends). No status guard prevents late submission from healing a Partial run to Completed. This means:

- Duplicate submissions are safe
- Late submissions (after an agent reconnects) heal partial runs
- The orchestrator never says "you're too late to submit this result"

Config: env vars override JSON

Config is loaded from `/etc/bas/config.json` first, then env vars override individual fields. The `config.Load()` function applies overrides in order. In Docker Compose deployments, all config comes from env vars (no JSON file needed).

Adding a New API Endpoint

1. **Define handler** in the appropriate `internal/api/` file (or create a new file for a new resource, e.g. `internal/api/threat_intel_config_handlers.go`)
2. **Register route** in `orchestrator/internal/api/routes.go` — **not** `main.go`. Routes are registered with `auth.RequirePermission(...)` middleware inline, e.g.:

   ```
   r.With(auth.RequirePermission(auth.CanUpdateConnecto
   ```

3. **Auth check:** Add a `Permission` constant in `internal/auth/permissions.go` if an existing one doesn't fit (const block, `rolePermissions[Role...]` map, `Permissions()` enumeration, and the corresponding rows in `permissions_test.go`'s `TestHasPermission_FullMatrix`/`TestHasPermission_MatrixIsComplete`/`TestPermissions_Ordering` — **never** touch `TestPermissionGrants_MatchMigrationInventory`, which is a frozen historical snapshot of the original RBAC migration and must not grow for permissions added after it).
4. **Input validation:** Validate IDs with regex; validate JSON with `json.Decoder`. Return `400` via `jsonError(w, "...", http.StatusBadRequest)` on invalid input.
5. **Response format:** Always return `application/json` via

`respond(w, ...)`. Use `jsonError(w, message, code)` for error responses.

6. **Document** in `docs/guides/api-reference.md`

Adding a New WebSocket Message Type

1. Define the message type constant in `orchestrator/internal/models/schema.go`:

```
const MsgMyNewEvent = "my_new_event"
```

2. Define the payload struct (if needed):

```
type MsgMyNewEventPayload struct {
    ResourceID string `json:"resourceId"`
    // ...
}
```

3. Broadcast via the hub:

```
hub.BroadcastJSON(models.WSMessage{
    Type: models.MsgMyNewEvent,
    Data: MsgMyNewEventPayload{...},
})
```

4. Handle in the dashboard JS (`orchestrator/wwwroot/index.html`):

```
case 'my_new_event':
  handleMyNewEvent(msg.data);
  break;
```

5. **Keep JS string literals in sync with Go constants.** There is no code-gen between Go and JS — they are coupled by convention.

Database Schema Changes

There is no numbered-migration-file system. Schema is defined as a single ordered `[]string` of idempotent raw SQL statements inside `EnsureSchema` in `orchestrator/internal/db/postgres.go` (plus a few sibling functions — `EnsureContentSchema`, `EnsureExerciseSchema` — for schema that loads separately). This runs in full on every orchestrator startup.

Writing a schema change:

- Add a new `CREATE TABLE IF NOT EXISTS ...` (or `ALTER TABLE ... ADD COLUMN IF NOT EXISTS ...` for an existing table) entry to the `[]string` slice inside `EnsureSchema` — near a related existing table for locality, exact position within the slice doesn't matter
- Every statement must be idempotent — `IF NOT EXISTS` on every `CREATE`/`ADD COLUMN`, since this runs unconditionally on every boot, not just once
- Never rewrite a previously-shipped statement in a way that would fail against a database that already has that column/table (e.g. don't change a `CREATE TABLE`'s column list after it's shipped — add a new `ALTER TABLE ADD COLUMN IF NOT EXISTS` instead)
- **Also add it to `orchestrator/internal/testutil/testdb.go`** if it's a new `EnsureXSchema` function (not needed if you're adding to the existing `EnsureSchema`/`EnsureContentSchema`/`EnsureExerciseSchema` calls, since the test harness already calls those) — otherwise every handler test against the shared test DB hits "relation X does not exist". This is a real, easy-to-miss gotcha; it silently doesn't affect production, only tests.
- Test against the real container-backed test DB (`go test ./...`

— see Testing below) before release; there's no separate "fresh database" migration-order concern to worry about since there's no ordering, just one full idempotent pass every boot

---

Testing

```
cd orchestrator
go test ./... -p 2            # Windows/Docker Desktop de
go test ./... -race           # Race detector
go test -cover ./...          # Coverage
```

Handler tests use `httptest.NewRecorder()` and `httptest.NewRequest()`. Database-dependent tests spin up a real, throwaway PostgreSQL instance automatically via `testcontainers-go` (Docker must be running) — no `DATABASE_URL` to set, no mock database. Pass `-short` to skip container-backed tests when Docker isn't available. Most packages share one container per test binary (`sharedDB`/`MustSharedTestDB()`, truncated between tests) rather than spinning up a fresh container per test — but each *package* still gets its own container, and `go test ./...` runs multiple packages' test binaries concurrently by default.

Windows dev host memory note: with no `-p` flag, `go test ./...` defaults to Go's full CPU-count parallelism, which on a memory-constrained Docker Desktop host (this build host has ~7.8GB total RAM, Docker capped at ~3.8GB) starts too many Postgres containers at once and fails with spurious `unexpected EOF` Postgres-connection errors — not a real test/code defect. `-p 2` is the validated safe ceiling on this host (confirmed clean under a full cold run, ~35-45% faster than fully serial), but only once enough memory is free — stop `gopls.exe` first (`taskkill /F /IM gopls.exe`; it restarts automatically, no data lost) if free RAM is under ~1.5GB. If `-p 2` still throws `unexpected EOF` errors, fall back to `go test ./... -p 1` (fully serial, always safe, just slower). This is a local-machine-only workaround — CI (`ubuntu-latest` GitHub-hosted runner) runs bare `go test ./...` with no `-p` and has not shown this problem, since it's a different OS/memory profile

without Docker Desktop's VM overhead.

Common Development Pitfalls

Pitfall	Description
Template field not found at runtime	Struct field obfuscated by garble
WebSocket message not received in browser	JS string literal doesn't match Go constant
Score returns 0	All steps ERROR or SKIPPED; denominator is 0
Agent binary trust all yellow	`BINARIES.sha256` not updated after agent binary chang
New table/ `EnsureXSchema` not visible in tests	`internal/testutil/testdb.go`'s test harness has its own hand-maintained list of `db.Ensure*Schema` calls, separate from production startup

New permission missing from RBAC tests	`permissions_test.go` has 3 test sites to update (`TestHasPermission_FullMatrix`, `TestHasPermission_MatrixIsComplete`, `TestPermissions_Ordering`) plus one to **never** touch (`TestPermissionGrants_MatchMigrationInvento` a frozen historical snapshot)
`jwt_secret required` on startup	`.env` file not loaded by Docker Compose
Config file secrets in logs	`jwt_secret/agent_secret` loaded from JSON file

Release Checklist

- ☐ There is no `version.go` to edit — the version string is passed as `-Version` to `packaging\windows-build.ps1` and flows from there into the Docker build-arg, `dist\bas-install-<version>\`, and the ZIP name. Nothing to hand-edit beforehand.
- ☐ Run `go mod tidy`
- ☐ Run `go test ./... -p 2` (or `-p 1` if that throws `unexpected EOF` errors — see Testing above) — all green
- ☐ Run `packaging\windows-build.ps1 -Version <x.y.z> -Customer ... -CustomerID ... -Days ...` — full pipeline
- ☐ Scenario/manifest signature verification is automatic at orchestrator startup (RSA-4096, public key compiled into the binary) — there's no separate pre-release CLI verify step; a build that produced unsigned or mismatched `.sig` files will fail to load those scenarios when the new image starts, which is itself the check
- ☐ If GPG bundle-signing is configured, confirm the build's summary output reports the ZIP as signed (`.zip.asc` present), not skipped with a warning
- ☐ Smoke-test the delivery ZIP on a clean Ubuntu VM via

`install.sh --install`

- ☐ Update `docs/guides/release-notes.md`
- ☐ Git tag: `git tag v1.7.5 && git push --tags`

© Audspect Engineering — Internal / Confidential